WEEKLY MINI PROJECT 04 · TEAM 3

Bytes for Futures Assessment Platform

Work Package · Leader / Frontend A

Leader | Frontend | React + TS | Integration

PROJECT	TEAM LEADER	OWNER
Bytes for Futures Assessment Platform	**Ali Mikdad**	**Ali Mikdad**

ROLE	AREA
Team leader · Frontend developer	**Foundation · Auth · Instructor screens · Exam guard · Review**

> *In Phase 0 you unblock everyone. After that you build the instructor's side, and you bring every branch together.*

> **Use the project pattern:** `pages/` for screens, `context/` for server data, `services/` for API calls (only contexts call them), `types/` copied from `API.md`. Pages never call `fetch`, and never decide scores, locks or time.

> **Full reference:** this package is a summary of `docs/TASKS.md`. Every name, endpoint, shape and message is fixed in `docs/API.md` and `docs/PROJECT-STRUCTURE.md`. If this package and the docs ever disagree, the docs win; tell Ali.

1 Your Work at a Glance

#	Task	Main file(s)
T1	Repository and team rules (Phase 0)	`.github/`, `.gitignore`, `.editorconfig`, `.prettierrc`, `docs/`
T2	Frontend skeleton (Phase 0)	`client/`, `routes/`, `services/`, `types/`, `context/`, `exam-guard/`
T3	Monaco bundled locally (Phase 0)	`editor/monacoSetup.ts`
T4	Shared UI (first small PR)	`components/common/`, `utils/format.ts`
T5	Sign in / Sign up	`pages/Auth/`, `context/AuthContext.tsx`
T6	Test Management	`pages/instructor/TestManagement/`, `context/InstructorTestsContext.tsx`
T7	Test Editor	`pages/instructor/TestEditor/`
T8	Attempts (with the violation log)	`pages/instructor/Attempts/`, `context/InstructorAttemptsContext.tsx`
T9	Exam-guard toolkit	`client/src/exam-guard/`

#	Task	Main file(s)
T10	Review, integration and the team	GitHub
T11	README and presentation	`README.md`

2 Work Without Waiting

PHASE 0 · DAY 1 MORNING

Stubs everywhere

Ali and Esmaeel push both skeletons. Every endpoint answers with the example JSON from `API.md`; every route shows a placeholder.

PHASE 1 · DAYS 1–2

Build alone

Everyone works in their own files, against the fixed names. Replace stubs only inside your own module.

PHASE 2 · END OF DAY 2

Real integration

Every stub replaced, every context on the real endpoints, every message checked against `API.md`.

Your work depends on `docs/API.md` and `docs/PROJECT-STRUCTURE.md`, not on someone's unfinished code. If a dependency is not merged yet, code against the fixed name and test with the stub.

3 Your Tasks, Step by Step

T1 Repository and team rules (Phase 0, ~3 h with T2–T3)

`.github/CODEOWNERS` · `.github/pull_request_template.md` · `.gitignore` · `.editorconfig` · `.prettierrc` · `docs/`

1. Create the GitHub repository and add everyone.
2. Create the folders from `PROJECT-STRUCTURE.md`.
3. Add `.gitignore` (`node_modules`, `dist`, `.env`), `.editorconfig`, `.prettierrc`, a README stub, and the three docs.
4. Add `.github/CODEOWNERS` and `.github/pull_request_template.md` (`TASKS.md` section 15). Use the real GitHub usernames.
5. First commit directly on `main` (one-time exception), push, then **turn on branch protection right away**: require a PR, 1 approval, dismiss old approvals, block force pushes, restrict deletions, no bypassing (also for admins).

✔ Done when: branch protection is on and the team is told: "branch from `main` now".

T2 Frontend skeleton (Phase 0)

`client/src/routes/` · `client/src/services/` · `client/src/types/` · `client/src/context/` · `client/src/components/` · `client/src/exam-guard/`

1. Create the Vite + React + TypeScript project in `client/`. Install `react-router-dom`, `monaco-editor`, `@monaco-editor/react`.
2. `styles/fonts.css` (local font files or system fonts), `styles/theme.css`, `styles/global.css`.
3. `services/http.ts` (`request<T>()`, `ApiError`, token header, `setUnauthorizedHandler`; `options.headers` are merged in) and the three `*.api.ts` files with every function from `PROJECT-STRUCTURE.md`, section 4.5.
4. All `types/*.types.ts` files from `API.md`, section 5.
5. `AuthContext.tsx` fully written. The other four contexts as **skeletons**: Provider, hook, fixed value shape, empty default actions.
6. `App.tsx` with the five providers. `routes/AppRouter.tsx` with **all routes** pointing to placeholder pages. `RequireRole.tsx`.
7. `components/layout/` (Layout, Navbar) and placeholder `components/common/` with the exact props (including a placeholder `Countdown`).
8. Placeholder `exam-guard/` files with the exact exports from `PROJECT-STRUCTURE.md`, section 16.2 (hooks that do nothing, components that render their children), so Maya can wire them from day one.
9. `client/.env.example` with `VITE_API_URL=http://localhost:3000/api`.

✔ Done when: `npm run dev` starts and every route shows its placeholder.

T3 Monaco bundled locally (Phase 0)

`client/src/editor/monacoSetup.ts` · `client/src/main.tsx`

1. `@monaco-editor/react` loads Monaco from a CDN by default. Switch that off with the file below, so the editor works in Syria without a VPN.
2. `main.tsx` imports `./editor/monacoSetup` once.

```
// client/src/editor/monacoSetup.ts
import * as monaco from 'monaco-editor';
import { loader } from '@monaco-editor/react';
import editorWorker from 'monaco-editor/esm/vs/editor/editor.worker?worker';
import tsWorker from 'monaco-editor/esm/vs/language/typescript/ts.worker?worker';

self.MonacoEnvironment = {
  getWorker(_workerId: string, label: string) {
    if (label === 'typescript' || label === 'javascript') return new tsWorker();
    return new editorWorker();
  },
};

loader.config({ monaco });   // use the bundled copy, never the CDN

monaco.languages.typescript.javascriptDefaults.setDiagnosticsOptions({
  noSemanticValidation: false,
  noSyntaxValidation: false,
});
monaco.languages.typescript.javascriptDefaults.setCompilerOptions({
  target: monaco.languages.typescript.ScriptTarget.ES2020,
  allowJs: true,
  checkJs: true,
  lib: ['es2020'],             // no DOM: students write plain functions, no `document`
  allowNonTsExtensions: true,
});

// lib is es2020 only, so `console` is unknown and would be underlined red. Declare it for students' debug
logs.
monaco.languages.typescript.javascriptDefaults.addExtraLib(
  'declare const console: { log(...data: any[]): void; error(...data: any[]): void; warn(...data: any[]):
void; info(...data: any[]): void };',
  'ts:b4f-console.d.ts',
);
```

✔ Done when: the editor opens with **no request to any CDN** in the Network tab.

T4 Shared UI (first small PR, right after Phase 0)

`client/src/components/common/` · `client/src/utils/format.ts`

1. Replace the placeholders with the real `Loading`, `ErrorMessage`, `EmptyState`, `Button`, `StateBadge`, `ConfirmDialog`, `PageContainer`. Props are fixed (`PROJECT-STRUCTURE.md`, section 4.3).
2. `StateBadge` colors: `AVAILABLE` and `PUBLISHED` → primary, `IN_PROGRESS` → warning, `SUBMITTED` → success, `EXPIRED` → error, `TERMINATED` → terminated, `LOCKED` and `DRAFT` → muted. The text label is always shown.
3. `utils/format.ts` (table below). Merge this first: Maya imports them.

Function	Example
`formatDateTime(iso)`	`2026-11-03T10:00:00Z` → `3 Nov 2026, 13:00` (local time)
`formatDuration(seconds)`	`1020` → `17 min 0 s`
`formatScore(score)`	`80` → `80%`, `72.5` → `72.5%`, `0` → `0%`, `null` → `Expired — not submitted`

✔ Done when: Shared UI is merged and used by Maya.

T5 Sign in / Sign up

`pages/Auth/SignInPage.tsx` · `pages/Auth/SignUpPage.tsx` · `context/AuthContext.tsx` · `routes/RequireRole.tsx`

1. `SignInPage`: email, password, **Sign in**. `SignUpPage`: full name, email, password, **Create account**.
2. Check only empty fields before sending (`Please enter your email.` etc.). Every other rule comes from the backend message.
3. Buttons disabled while loading. Errors shown inside the form.
4. After success, go to `/` (redirect by role).
5. Signed-in users who open `/sign-in` are sent to `/`.
6. **Sign out** in the Navbar clears the token and navigates to `/sign-in`.
7. `AuthContext`: on start, if a token is saved, call `getMe()`; a `401` clears the token. `http.ts` calls `onUnauthorized()` on every `401`, so an expired token signs the user out everywhere.

✔ Done when: sign in, sign up, and sign out work for both roles (sign out goes to `/sign-in`); an expired token signs the user out; protected routes show a sign-in prompt without a valid token.

T6 Test Management (`/instructor/tests`)

`pages/instructor/TestManagement/` · `context/InstructorTestsContext.tsx`

1. `loadTests()` when the page opens.
2. One `TestRow` per test, in order: title, type, duration, status badge, question/problem count, attempt count, **Publish** or **Unpublish**, **Edit content**.
3. Publish/unpublish: button loading while the request runs. A `409` shows the backend message next to that row. The row updates without a page reload.

✔ Done when: Test Management shows the three tests and publishes/unpublishes with backend messages for rejections.

T7 Test Editor (`/instructor/tests/:testId`)

`pages/instructor/TestEditor/`

1. `loadTest(testId)` when the page opens.
2. MCQ test: `QuestionList` (each question with its options, the correct one marked) + `QuestionForm` for add and edit.
   - Text, 2–6 options, a radio for the correct option, **Add option** / remove option.
   - Front-end guidance only: Add option disabled at 6, remove disabled at 2.
3. Coding test: `ProblemList` + `ProblemForm`: title, description, function name, points, and test cases (`TestCaseFields`: visibility select, args JSON, expected JSON, remove).
   - Args and expected are typed as JSON. Invalid JSON shows `Please enter valid JSON.` and is not sent.
   - Helper text: `Arguments are a JSON array, for example [[2, 7, 11, 15], 9].`
4. Delete asks for confirmation (`ConfirmDialog`).
5. `isContentLocked` → every edit control disabled and the note `Content is locked because a student has started this test.`
6. Backend `400` / `409` messages show inside the form. What the instructor typed is kept.

✔ Done when: the editor adds, edits, and deletes questions and problems, and is locked when `isContentLocked` is true.

T8 Attempts (`/instructor/attempts`)

`pages/instructor/Attempts/` · `context/InstructorAttemptsContext.tsx`

1. `AttemptsFilters`: test select, status select (including `TERMINATED`), search box (search sends after the user stops typing for 400 ms, or on Enter).
2. Filters live in the URL query (`?testId=&status=&search=`), so a refresh keeps them.
3. Table: student, test, status badge, started, submitted, time used, score (`formatScore`), violations count.
4. Clicking a row opens `AttemptDetails` (MCQ correct count, or per-problem runs / submitted / passed / points), plus the termination reason and the violation log (type, time, counted or not).
5. Loading, empty (`No attempts match these filters.`), error with retry.

✔ Done when: the Attempts page filters, searches, and shows details. Expired attempts show `Expired — not submitted`. Terminated attempts show their score, the reason, and the violation log.

T9 Exam-guard toolkit (`client/src/exam-guard/`)

`useFullscreen.ts` · `useExamGuard.ts` · `editorPasteGuard.ts` · `FullscreenGate.tsx` · `ViolationWarningDialog.tsx` · `ExamGuard.css`

Build the hooks and components from `PROJECT-STRUCTURE.md`, section 16.2. They know nothing about the API: they detect, block, and call `onViolation(type)`.

1. `useFullscreen()`: `isSupported` (`document.fullscreenEnabled`), `isFullscreen` (from `fullscreenchange`), `enter()` (returns `false` if the browser refused; waits at most 3 s with `Promise.race`, then returns `!!document.fullscreenElement`, because some browsers never settle the promise), `exit()`.
2. `useExamGuard({ isActive, onViolation })`:
   - leaving full screen → `FULLSCREEN_EXIT`; page hidden → `TAB_HIDDEN`
   - `copy` / `cut` outside `[data-exam-clipboard="allow"]` → `preventDefault()` + `COPY` / `CUT`
   - `contextmenu` outside the editor → `preventDefault()` only (not reported)
   - events of the same group within 1 s → sent once
   - every listener removed when `isActive` becomes `false` and on unmount
3. `attachEditorPasteGuard(editor, onViolation, root)`: remember text copied or cut inside the editor (`editor.getModel().getValueInRange(selection)`); on `paste` (capture phase on `root`, the wrapper `div` that `CodeEditor` passes with a `ref` — **not** `editor.getDomNode()`, which Monaco can re-create so the listener is lost), if `clipboardData.getData('text/plain')` is not the remembered text → `preventDefault()`, `stopPropagation()`, `PASTE_EXTERNAL`; on `drop` from outside → block + `DROP_EXTERNAL`. Returns `dispose()`.
4. `FullscreenGate` (`isFullscreen`, `onEnter`, `children`) and `ViolationWarningDialog` (`warning`, `onReturn`) with the fixed props and the `exam-guard-` CSS prefix. In your test route, put the dialog **outside** the gate (the gate hides its children when full screen is left).
5. Check every event by hand in Chrome, Edge and Firefox with a temporary test route on your own branch. Remove the route before opening the PR.

✔ Done when: the toolkit reports every type in `API.md` 19.1, blocks copy, cut, external paste and external drop, allows copy and paste inside the editor, and removes its listeners when inactive.

T10 Review, integration and the team

`GitHub`

Review

- Review every PR, normally within **2 hours**.
- Check: only owned files changed, names and CSS prefixes follow the docs, the PR template is filled, the build works.
- Ask for a split when a PR is too big.

Integration

- Merge in dependency order (`TASKS.md` section 17).
- After every merge, `main` must still start and build.
- Run the full test (`TASKS.md` section 12) at the end of Phase 2 and again before the demo.

Team

- 15-minute daily sync: done, next, blockers.
- One GitHub issue per Definition-of-Done item.
- Decide contract changes and update `docs/` in the same PR.
- Ask the instructor about Judge0 and Monaco (`PROJECT-STRUCTURE.md`, section 3) on Day 1. Link the written approval for exam integrity there.
- Freeze `main` two hours before the demo.

✔ Done when: all PRs are reviewed and the full test passed on a clean clone.

T11 README and presentation

`README.md`

A clean checkout must run by following the README only. It explains:

- what the project is and the three tests
- the technologies
- the folder structure
- requirements (Node 18+, PostgreSQL)
- how to set up the database (`.env`, `npx prisma migrate deploy`, `npx prisma db seed`)
- how to run the backend and open Swagger
- how to run the frontend
- the environment variables (`VITE_API_URL`, `DATABASE_URL`, `JWT_SECRET`, `EXECUTION_MODE`, Judge0 variables)
- execution modes (mock and Judge0) and how to switch
- seed accounts (local development only)
- the business rules in short (order, timing and grace, expiry, scoring, Run vs. Submit)
- a note that everything works without a VPN, and what to do if the execution host is down
- the team and who built what

Presentation: plan the speakers from `TASKS.md` section 14, run two full rehearsals with a timer.

✔ Done when: README is complete and two full rehearsals are done.

4 Fixed Shapes You Own

Shapes from `PROJECT-STRUCTURE.md`, section 4.2. Maya copies the pattern of `AuthContext`.

client/src/context/*.tsx (fixed value shapes)

```
// AuthContext (Ali)
user: User | null;
isAuthReady: boolean;               // false until the saved token was checked with /auth/me
authError: string | null;
isAuthLoading: boolean;
signIn(payload: LoginPayload): Promise<boolean>;
signUp(payload: RegisterPayload): Promise<boolean>;
signOut(): void;

// InstructorTestsContext (Ali)
tests: InstructorTestSummary[];
isTestsLoading: boolean;
testsError: string | null;
loadTests(): Promise<void>;
currentTest: InstructorTestDetail | null;
isTestLoading: boolean;
testError: string | null;
loadTest(testId: number): Promise<void>;
isSaving: boolean;
actionError: string | null;
publishTest(testId: number): Promise<boolean>;
unpublishTest(testId: number): Promise<boolean>;
saveQuestion(testId: number, payload: QuestionPayload, questionId?: number): Promise<boolean>;
deleteQuestion(questionId: number): Promise<boolean>;
saveProblem(testId: number, payload: ProblemPayload, problemId?: number): Promise<boolean>;
deleteProblem(problemId: number): Promise<boolean>;
clearActionError(): void;

// InstructorAttemptsContext (Ali)
attempts: InstructorAttemptRow[];
isAttemptsLoading: boolean;
attemptsError: string | null;
loadAttempts(filters: AttemptFilters): Promise<void>;
selectedAttempt: InstructorAttemptDetail | null;
isDetailLoading: boolean;
detailError: string | null;
loadAttemptDetail(attemptId: number): Promise<void>;
```

Component	Props
`Loading`	`message?: string`
`ErrorMessage`	`message: string`, `onRetry?: () => void`
`EmptyState`	`title: string`, `description?: string`
`Button`	`variant?: 'primary' | 'secondary' | 'danger' | 'ghost'`, `isLoading?: boolean`, plus normal button props
`StateBadge`	`state: StudentTestState | AttemptStatus | TestStatus`
`ConfirmDialog`	`isOpen`, `title`, `message`, `confirmLabel`, `onConfirm`, `onCancel`, `isLoading?`
`PageContainer`	`title?: string`, `actions?: ReactNode`, `children`

5 Rules for You

- Only edit your own files (`TASKS.md` section 4).
- Do not build student screens. Do not write backend code.
- Do not decide scores, locks, or time on the frontend.
- Do not merge a PR that nobody reviewed. Maya approves your frontend PRs; Esmaeel your `docs/`-only PRs.
- Do not change a fixed name silently.
- CSS uses the `auth-`, `test-mgmt-`, `test-editor-`, `attempts-`, `not-found-`, `ui-`, `layout-`, `exam-guard-` prefixes and theme colors only.

- Loading, empty, and error states on every page. No `fetch` inside pages or components.

6 Presentation Part

Your parts in the English presentation: **the problem and the product idea**, and **a decision we made and why (Judge0 behind the backend, no CDN for Syria) and one limitation / next step**. Prepare your part and one rule from another member's area, and be ready to explain your own code.